U.PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

Job Shop and Open Shop Scheduling

MECD – FEUP 2025/26

Course - ANALYTICAL DECISION SUPPORT SYSTEMS

Beatriz Sonnemberg, up202206098

Maria Beatriz Moreira, up202208293

Marta Costa, up202207879

Problem Description

For both MIP and CP implementations:

- **Job Shop Scheduling (JSP):** operations within each job follow a strict technological sequence.

- **Open Shop Scheduling (OSP):** no fixed order is imposed, although operations of the same job cannot overlap.

- **Goals:**
  - Model JSP and OSP using both MIP and CP
  - Minimize the **makespan**: completion time of all operations

Approaches Overview

2 Optimization Paradigms: MIP and CP

- **MIP:**
    - PuLP library
    - Classical disjunctive formulation based on **Big-M** constraints
    - Large number of binary variables required to model machine conflicts

- **CP:**
    - Google OR-Tools (CP-SAT solver)
    - High-level global constraints:
        - ❖ *NoOverlap* constraint – to handle machine and job conflicts more efficiently

MIP Model

- **Objective:** Minimize makespan
- **Time limit:** Ensures fair comparison with CP
- **Decision Variables:**
    - Continuous: start times of operations
    - Binary: sequencing decisions
- **Constraints:**
    - **Machine capacity:** disjunctive Big-M to prevent overlap
    - **Job precedence:** enforce technological order (JSP)
    - **Open Shop:** operations within the same job cannot overlap
- **Big-M:** set as sum of all processing times (planning horizon)

- **Outputs:**
    - Solution status
    - Makespan value
    - Solve time
    - Count of binary and continuous variables
- **Limitations:**
    - High number of binary variables and Big-M constraints
    - Branch-and-bound search becomes expensive
    - Poor scalability for large or Open Shop instances

CP Model

- **Objective:** Minimize makespan
- **Time limit**: Same limits as MIP for fair comparison

- **Search strategies**
  - Supports strategies: DEFAULT, FIXED_SEARCH, NO_PRESOLVE, RANDOMIZED
- **Decision variables:**
  - Interval variables for each operation (start, end, duration)
  - Integer variables for start and end times

- **Machine Constraints:**
  - NoOverlap constraints prevent machines from processing multiple operations simultaneously
  - Automatically handled via CP intervals, no need for Big-M
  - Efficiently manages all disjunctive constraints
- **Complexity:**
  - Interval-based model reduces the need for quadratic binary variables

- **Outputs:**
  - Solution status
  - Makespan
  - Solve time

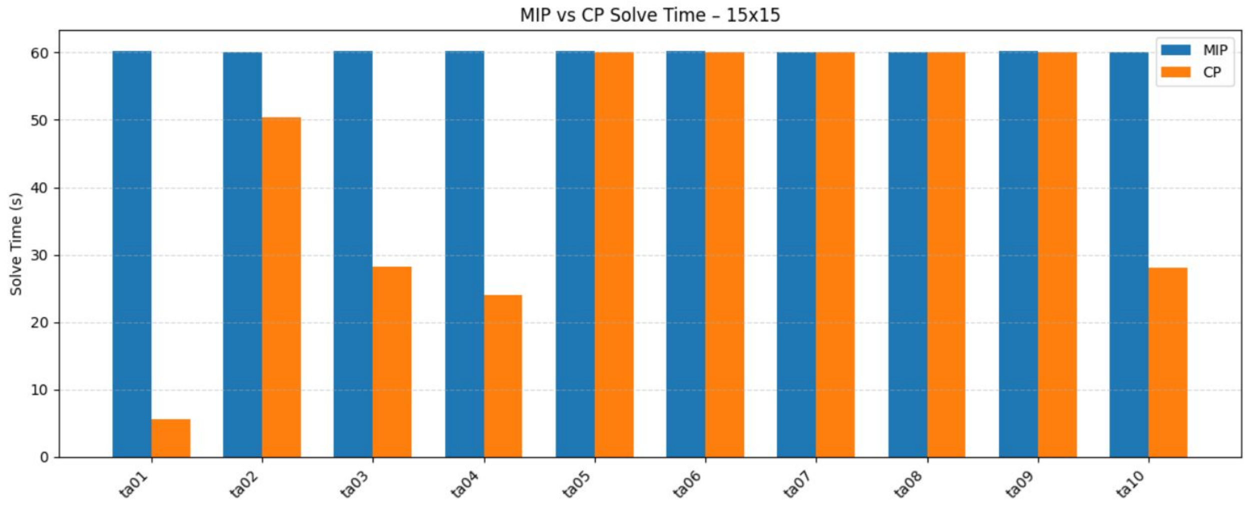
- **Search and Constraints:**
  - Precedence constraints enforce job order
  - Strong constraint propagation
  - Early pruning of infeasible schedules
  - Focuses on efficient search strategies, improving solution quality and computational time

MIP vs. CP: Experimental Setup

- Benchmark instances from the Taillard Job Shop Scheduling library
- 5 Experiences:
  - Experience 1: Instances 15×15 | Time limit = 60s
  - Experience 2: Instances 20×15 | Time limit = 90s
  - Experience 3: Instances 20×20 | Time limit = 120s
  - Experience 4: Instances 30×15 | Time limit = 150s
  - Experience 5: Instances 30×20 | Time limit = 180s
- Both **MIP** and **CP** models executed under identical time limits and conditions
- Validation performed on small instances with known optimal solutions to ensure model correctness

Results – Solving Time

- **MIP** reaches the time limit in most instances, even for small problem sizes
- **MIP** struggles with scalability due to the large number of binary variables
- **CP** finds feasible solutions much earlier and uses computational time more efficiently
- **CP** shows superior runtime behavior, especially in larger instances

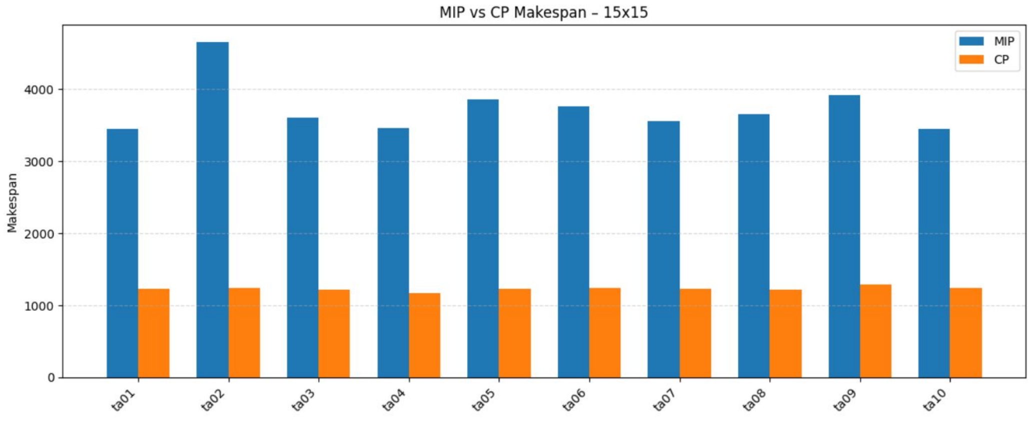


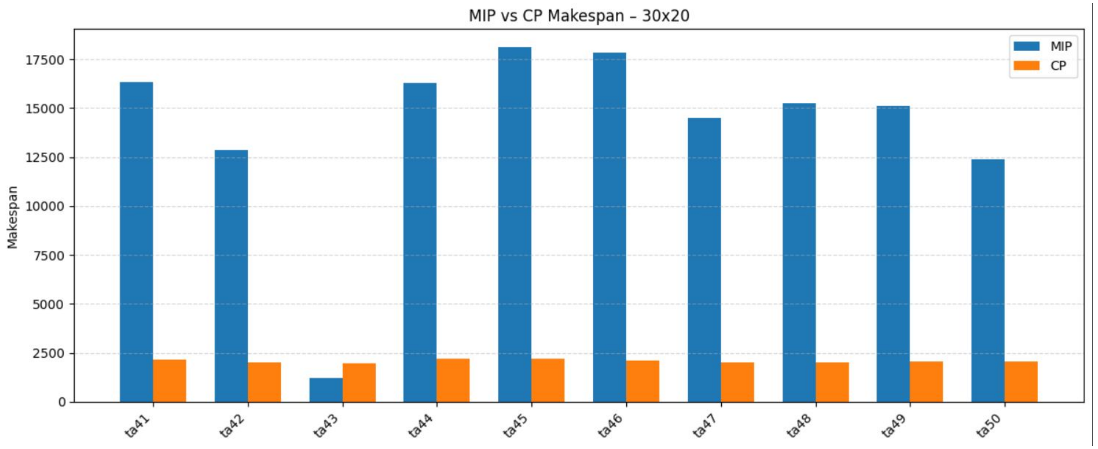
Results – Makespan Quality

- **CP** consistently achieves makespan values close to the Best Known Solutions (BKS)
- **MIP** produces significantly higher makespans, indicating larger optimality gaps

➔ As instance size increases, the gap between CP and MIP becomes more pronounced

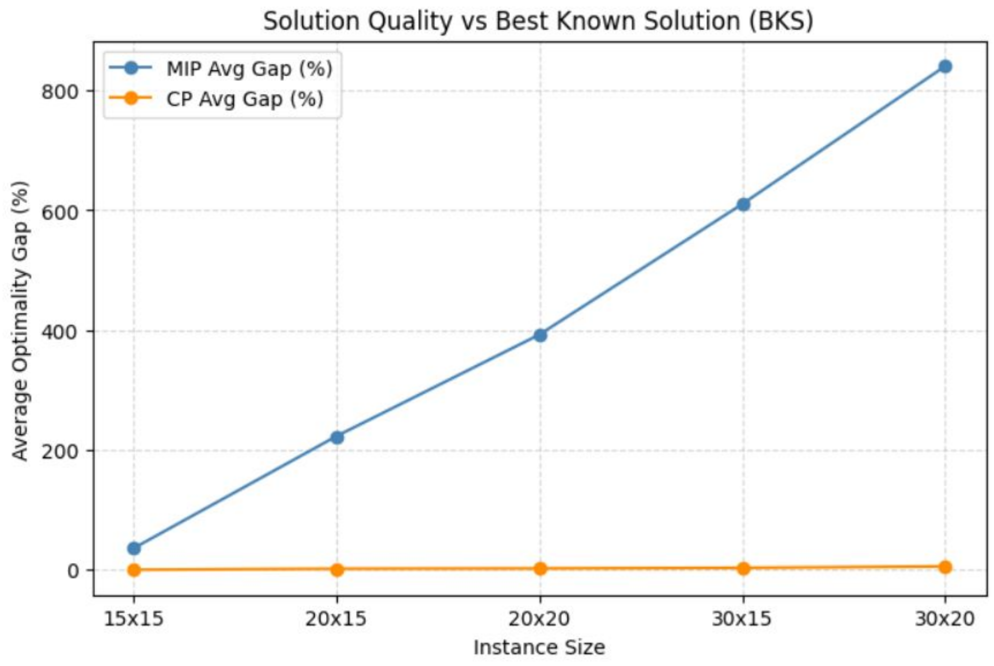
- CP maintains stable solution quality, while MIP struggles with larger instances





Optimality Gap Analysis

- CP stays close to Best Known Solutions (BKS)
- MIP makespans grow sharply as size increases



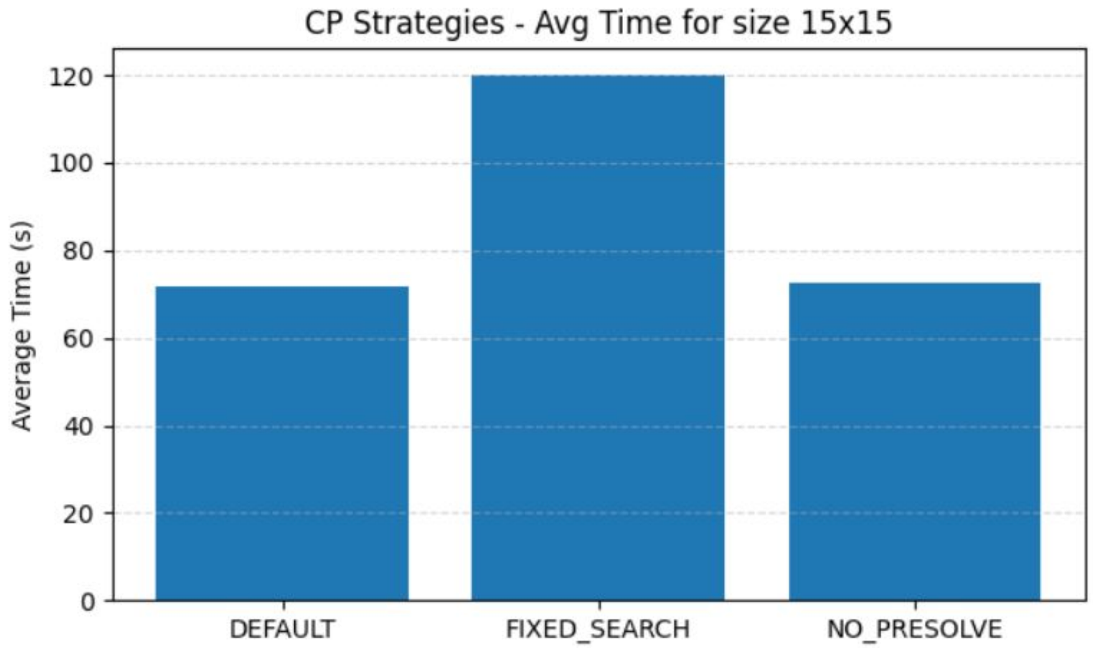
Sensitivity Analysis: CP Search Strategies

Strategies:

- **DEFAULT:** Best overall (uses adaptive SAT-based logic)
- **FIXED_SEARCH:** Slowest; shows that deterministic branching is inefficient
- **NO_PRESOLVE:** Highlights the importance of algebraic simplification

Instances Used:

- 10 instances 15x15
- 10 instances 20x15
- 10 instances 20x20



Open Shop vs Job Shop Analysis

1. Flexibility vs. Performance

- **Flexibility Gain:** Reordering operations eliminates forced idle time on machines.
- **CP Advantage:** Constraint Programming exploits this freedom effectively, filling gaps in the schedule that are otherwise locked in JSP.

2. The "Curse of Dimensionality" in MIP

- **Complexity Explosion:** Every possible sequence in OSP requires new binary variables.
- **Combinatorial Growth:** Modeling OSP in MIP doubles the disjunctive constraints, making it impractical for medium/large instances.

3. CP Scalability and Robustness

- **Efficient Search:** CP handles the expanded search space using the same AddNoOverlap primitives.
- **Stable Quality:** While MIP's performance collapses, CP maintains high-quality solutions even with increased flexibility.

- A Clear Trade-off: OSP provides better schedules (lower makespan) but creates extreme complexity for MIP
- CP remains the only robust solution for flexible industrial scenarios.

Conclusion

- CP consistently outperforms MIP in both Job Shop and Open Shop scheduling, especially as instance size increases
- CP exploits interval variables and global constraints to handle temporal and resource conflicts efficiently
- CP produces lower makespans and shows superior scalability compared to MIP
- Open Shop relaxation in MIP provides more scheduling flexibility but increases complexity
- Results reinforce CP as the preferred approach for large or flexible scheduling scenarios